



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

NASM Developer Environment on Windows

CMSC 131 Lab 0

Prepared by: Rene Andre Bedonia Jocsing

Published: August 24, 2025

CMSC 131 is often the first foray of a CS student into the dark domain of command line usage, workflow orchestration, and low-level programming. To alleviate frustration, the instructor has taken the liberty of diving into the abyss and charting a course.

The following steps require patience, a flexible adherence to instructions, and a good internet connection.

The goal is to prepare a developer environment for NASM X86 on Windows using Visual Studio Code, in preparation for the laboratory assignments that follow.

If you do not wish to use a Windows environment for the course, you may look into WSL, Linux, or DOSBox. But these methods will not be covered here, as plenty of documentation exists for them on the internet. Documentation is, however, scarce for Windows.

The course will rely heavily on Paul Carter's book: [PC Assembly Language](#) as reference (see: course materials). His [NASM library](#) will be useful for standardizing our laboratory assignments.

CMSC 131 has a steep learning curve as it introduces a paradigm distinct from other languages you may have encountered before like Python, Java, or C. Therefore, studying the course materials ahead of time is encouraged. The concepts and assignments in this course cannot be crammed overnight.

Installing NASM for Windows

1. Go to the [NASM](#) website.
2. Download the stable build by clicking on `stable` and navigating to the `/pub/nasm/releasebuilds/2.16.03/win64` directory.
3. Select the installer for download.
4. Run the installer and take note of where NASM is being installed on your system.
5. Go to your PATH (Edit the system environment variables).
6. Under `User Variables`, select `Path` and then `Edit`.
7. Select `New` then `Browse`.
8. Navigate to the directory from step 4 (default should be at `C:\Users\User\AppData\Local\bin\NASM`).
9. To verify installation, open a fresh command line and input the following:

```
1 nasm -v
```

You should see output like:

```
1 NASM version 2.16.03 compiled on Apr 17 2024
```

Installing GCC for Windows

1. Go to the [MinGW-64](#) website.
2. Open the Getting Started directory and select MSYS2 (GCC).
3. Follow the instructions on the MSYS2 website up to **Step 5**. This will install MSYS2 and update the core system libraries.
4. Once you see the MSYS2 terminal, **close** it, then go back to the page in step 2.
5. From here, follow the instructions on the aforementioned page. Additionally, install:

```
1 pacman -S mingw-w64-i686-toolchain
```

Verify that gcc is visible to Windows. Open a fresh command line and run:

```
1 gcc -v
2 gcc -where
```

If you find another (stray) GCC installation, edit your PATH so that C:/msys64/mingw32/bin/gcc.exe comes **first**. Also, if gcc -v shows a 64-bit toolchain, double-check that you are using the 32-bit variant (mingw-w64-i686-toolchain).

Fetching Paul Carter's Library for NASM

1. Go to the [pcasm](#) website.
2. Scroll down to the **Example Code** section.
3. Select **MS C Examples** and download the ZIP archive.
4. Extract the archive into a folder called pc_asm (or whatever you wish to name it).

Prepping Visual Studio Code for NASM Development

1. Open up **Visual Studio Code**.
2. Under the Extensions tab, search for NASM.
3. Install the **NASM X86 Assembly Language** extension.
4. Inside Visual Studio Code, navigate to the pc_asm folder from earlier.
5. To run first.asm, input the following lines one by one into the terminal:

```
1 nasm -f win32 first.asm -o first.obj
2 gcc -m32 -c driver.c -o driver.o
3 gcc --% -m32 first.obj asm_io.obj driver.o -o first.exe -Wl,-subsystem,console
4 ./first.exe
```

Let us examine what each command does in our manual build process:

The first command invokes NASM to **assemble** your assembly source file (first.asm) into a 32-bit **COFF object file** (first.obj), using the -f win32 flag.

The second command compiles the driver.c file (which calls your _asm_main) into a 32-bit object file. The -m32 ensures GCC targets 32-bit code, while -c tells it to compile but not link yet.

The third command links all three object files (`first.obj`, `asm_io.obj`, and `driver.o`) into a final 32-bit **Windows console executable**. The `-Wl,-subsystem,console` tells the linker that this is a command-line (not a GUI) program. `--%` is used in PowerShell to prevent argument parsing errors. You may try to omit this flag if you are using Windows CMD.

The final command runs your freshly built executable. If all went well, this should run your program in the terminal.

While typing these commands every time works, it is error-prone and tedious. That is why we will use a **task in Visual Studio Code** to automate and standardize this process. With one shortcut (`Ctrl + Shift + B`), Visual Studio Code runs all these steps for you reliably, across future assignments.

6. Create a `.vscode` folder inside the `pc_asm` folder.
7. Inside `.vscode`, create a `tasks.json` file.
8. Copy and paste the workflow from this [gist](#). *How this workflow was constructed from the previous commands is left as an exercise to you, the reader.*

For reference, here's the complete `tasks.json` content:

```

1 {
2   "version": "2.0.0",
3   "tasks": [
4     {
5       "label": "NASM Assemble (32-bit)",
6       "type": "shell",
7       "command": "nasm",
8       "args": [
9         "-f", "win32",
10        "${file}",
11        "-o", "${fileDirname}/${fileBasenameNoExtension}.obj"
12      ],
13      "group": "build"
14    },
15    {
16      "label": "GCC Compile Driver (32-bit)",
17      "type": "shell",
18      "command": "gcc",
19      "args": [
20        "-m32", "-c",
21        "${fileDirname}/driver.c",
22        "-o", "${fileDirname}/driver.o"
23      ],
24      "group": "build"
25    },
26    {
27      "label": "GCC Link (32-bit Console)",
28      "type": "shell",
29      "command": "gcc",
30      "args": [
31        "-m32",
32        "${fileDirname}/${fileBasenameNoExtension}.obj",
33        "${fileDirname}/asm_io.obj",
34        "${fileDirname}/driver.o",
35        "-o", "${fileDirname}/${fileBasenameNoExtension}.exe",
36        "--%",
37        "-Wl,-subsystem,console"
38      ],
39      "group": "build",
40      "dependsOn": [
41        "NASM Assemble (32-bit)",
42        "GCC Compile Driver (32-bit)"
43      ]
44    }
45  ]
46 }
```

```

44     },
45     {
46         "label": "Run Program",
47         "type": "shell",
48         "command": "${fileDirname}/${fileBasenameNoExtension}.exe",
49         "group": "test"
50     },
51     {
52         "label": "Build & Run",
53         "dependsOrder": "sequence",
54         "dependsOn": [
55             "GCC Link (32-bit Console)",
56             "Run Program"
57         ],
58         "group": {
59             "kind": "build",
60             "isDefault": true
61         }
62     }
63 ]
64 }

```

9. Save the reusable task.
10. Open `first.asm` and press `Ctrl + Shift + B`. You should see the program build and run.

Skeleton for Future Lab Exercises

1. In the `pc_asm` folder, create a subfolder named `skeleton`.
2. Extract the following files from the Paul Carter archive into this folder:
 - `asm_io.asm`
 - `asm_io.inc`
 - `asm_io.obj`
 - `cdecl.h`
 - `driver.c`
 - `skel.asm`
3. In the future, you may rename `skel.asm` to `lab1.asm`, `lab2.asm`, etc., for each lab.
4. Open the `skel.asm` file in Visual Studio Code.
5. The following shows how an introductory “Hello World” program looks like in assembly. *You may copy and paste the code from this [gist](#).*

```

1  ;
2  ; file: skel.asm
3  ; This file is a skeleton that can be used to start assembly programs.
4  ;
5
6  %include "asm_io.inc"
7
8  segment .data
9  ;
10 ; initialized data is put in the data segment here
11 ;
12 hello_msg db "Hello, world!", 0    ; null-terminated string for printing
13
14 segment .bss
15 ;
16 ; uninitialized data is put in the bss segment

```

```
17 ;
18
19
20 segment .text
21     global _asm_main
22 _asm_main:
23     enter    0,0                ; setup routine
24     pusha
25
26 ;
27 ; code is put in the text segment, but do not modify the code
28 ; before this comment.
29 ;
30
31     mov     eax, hello_msg      ; eax = "Hello, world!"
32     call    print_string       ; print(eax)
33
34     call    print_nl           ; print("\n")
35     call    print_nl           ; print("\n")
36
37 ;
38 ; return to C driver. Do not modify the code
39 ; after this comment.
40 ;
41     popa
42     mov     eax, 0              ; return back to C
43     leave
44     ret
```

6. Once the contents of your `skel.asm` match the code above, press `Ctrl + Shift + B` to build and run. You should see `Hello, world!` printed.

See the [laboratory manual](#) for submission requirements.